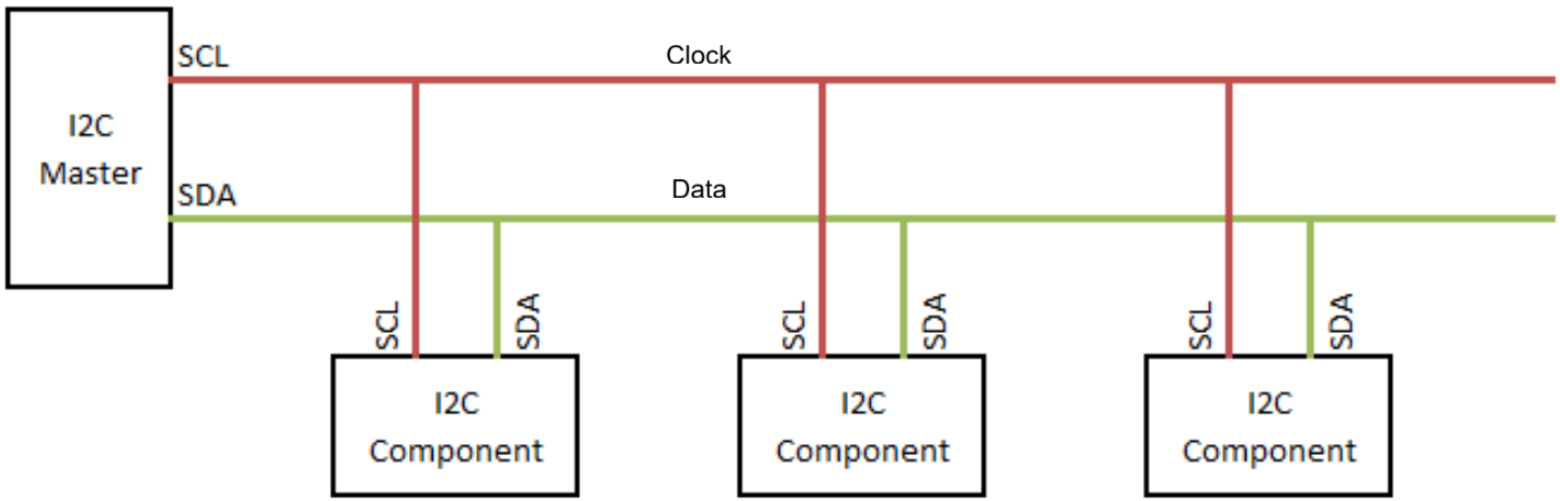
Serial Communication with I^2C

ECE 2564, Embedded Systems

Fall 2025

Overview

- I^2C: Inter-Integrated Circuit
- I^2C is another serial communication standard
  - Different from using a UART
- SCL: one-wire serial clock
- SDA: one-wire serial data
- I^2C is serial, synchronous, half-duplex communication



I^2C Modes

- A device can exist in one of two roles:
  - Master — controls the clock; addresses slave devices
  - Slave — receives the clock; responds to master
- Each role can involve *transmitting* and *receiving*
  - Scenario 1: Master sends address of a particular slave device, indicates "Write" and then starts transmitting data to that slave device
  - Scenario 2: Master send address of a particular slave device, indicates "Read" and then expects to receive data from that slave device

The I^2C Bus

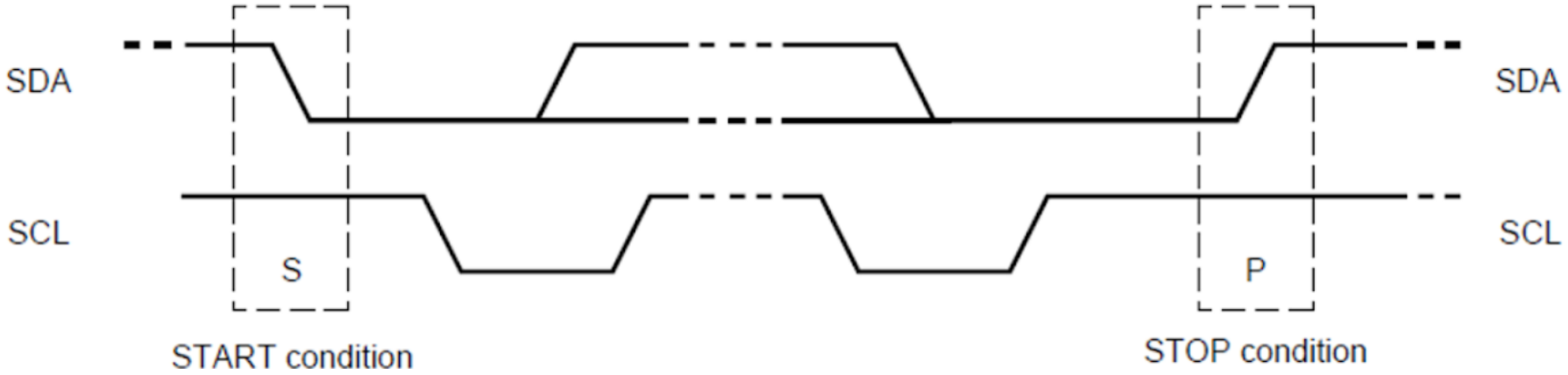
- I^2C is a *two-wire model* — it uses bi-directional *serial data* and *serial clock* lines
- Both lines are connected to the reference signal, V_{DD}, via pull-up resistors
- I^2C allows assigning the role of master device to multiple devices

I^2C Model

- Bus capacitance limits communications to a few meters
- Initially I^2C used a 7-bit address space, with 16 reserved addresses
- Initially I^2C operated at speeds of 10 kbit/s ("low speed") and 100 kbits/s ("standard")
- Currently other speeds are available:
  - 400 kbit/s ("fast mode")
  - 1 Mbit/s ("fast mode plus")
  - 3.4 Mbit/s ("high-speed mode")

Communication Protocol - 1

- Because there are only two wires, master/initiator had to signify *start* and *stop* conditions using these two wires
  - Start: SCL: high, SDA *transition* high to low
  - Stop: SCL: high, SDA *transition* low to high

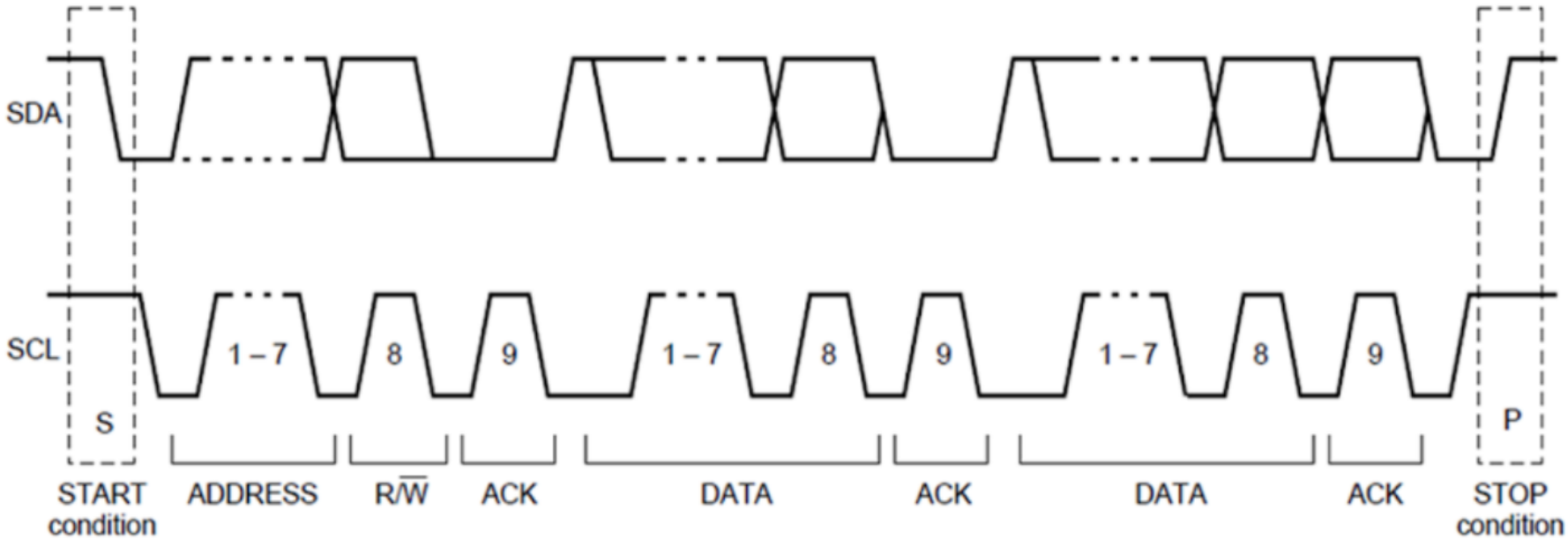


Communication Protocol - 2

- Master/initiator has to know the addresses for all peripherals and use the SDA to send the address
  - The address is 7 bits
    - How many addressable locations? $2^7 = 128$
    - The manufacturer determines the address of each device
  - The capacitance of the data and clock wires should not exceed $400\,\text{pF}$

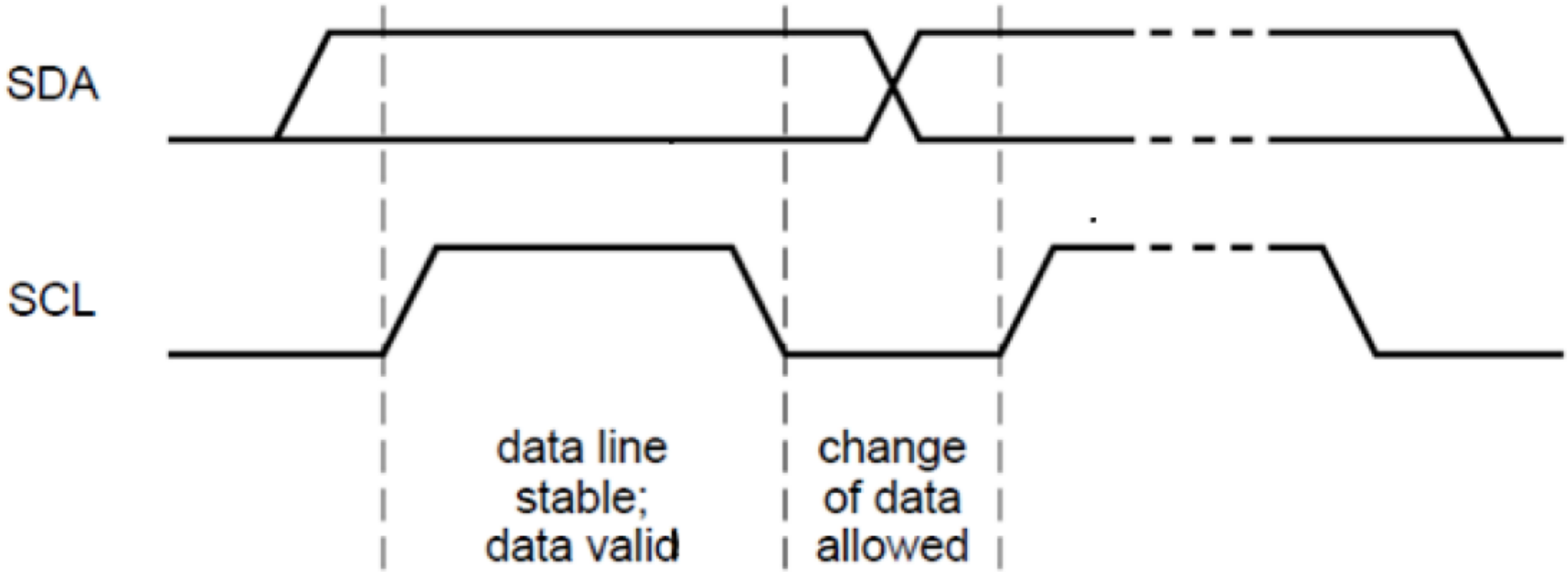
Communication Protocol - 3

- Master has to tell the slave device whether the master intends to read/listen or write/talk
  - NOTE: *active low* write
  - ACK: from the receiver

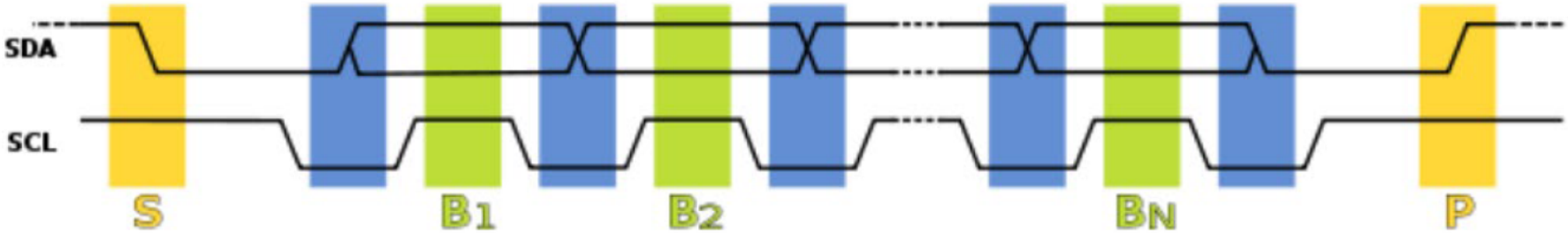


Communication Protocol - 4

- Whenever the SCL is high, the data should be held constant
- Whenever the SCL is low, the sender is allowed to change the data bit
  - Receiver reads the data at the falling edge of SCL



I^{2}C Communication Protocol - Recap

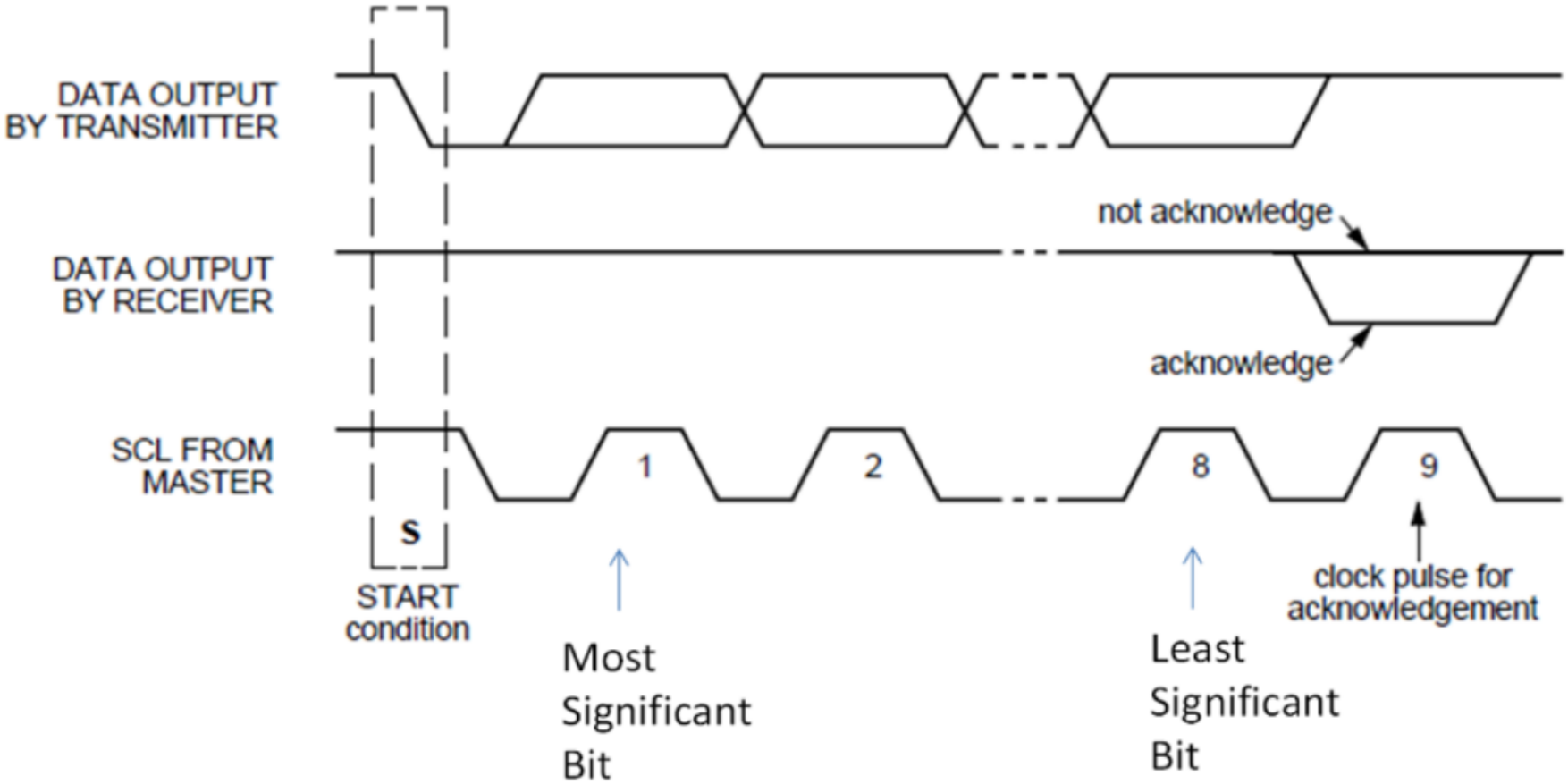


- SDA: serial data line
- SCL: serial clock
- S: start condition — pull SDA low while SCL is high
- B_i data bits — data is transmitted MSB *first* and is safe after the *rising (low-to-high transition) clock edge*
- P: stop condition — release SDA while SCL is high

I^2C Endianness

In I^2C protocol, the most significant bit (MSB) is sent first

- UART can be configured as to whether the MSB or LSB is sent first



I^2C Communication Review: The Process

1. A master takes control of the bus by signaling a start condition
2. The master sends the address of the slave device that the master wishes to engage
   - Master sends a byte containing a seven-bit address and a read/write bit (Read $= 1$ and Write $= 0$)
3. Each of the slave devices on the bus should be "listening" for its address; if the addressed slave is on the bus, it responds with an ACK bit by pulling SDA low for one clock cycle.

I^2C Communication Review: The Process

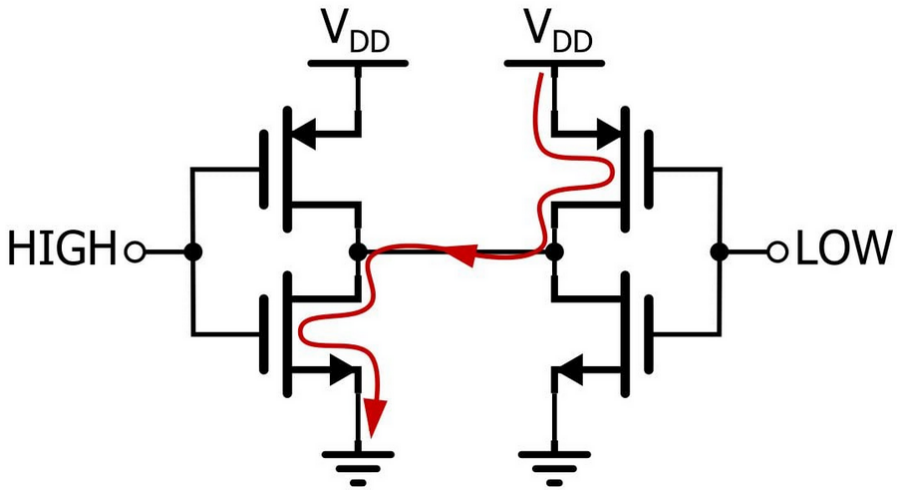
4. After the ACK bit, the master and slave continue in the roles established by the master.
   - Master transmit
     - Master sends a byte, slave returns an ACK
     - This repeats until the master stops sending bytes
     - The slave must acknowledge every byte it receives
   - Master receive
     - Slave sends a byte
     - Master returns an ACK on every byte except the last
     - The master sends a NACK to signal that it no longer wishes to receive bytes from the slave
5. To end the transmission, the master signals a stop condition (by releasing SDA) to end the transmission

“Failure to Communicate”

- After each byte, the sender and receiver (temporarily) “reverse roles”so the receiver can transmit its ACK
- But what does it mean if the ACK doesn’t come? It depends . . .
  - If the master was transmitting and the slave does not ACK, the master “knows” that the slave — for whatever reason — is unable to receive the data; this is because an I^2C slave is obligated to acknowledge each byte
  - If the slave was transmitting and the master does not ACK, the slave “knows” the master no longer wishes to receive data (as the programmer, you get to control the master when it comes to sending ACKs and NACKs)

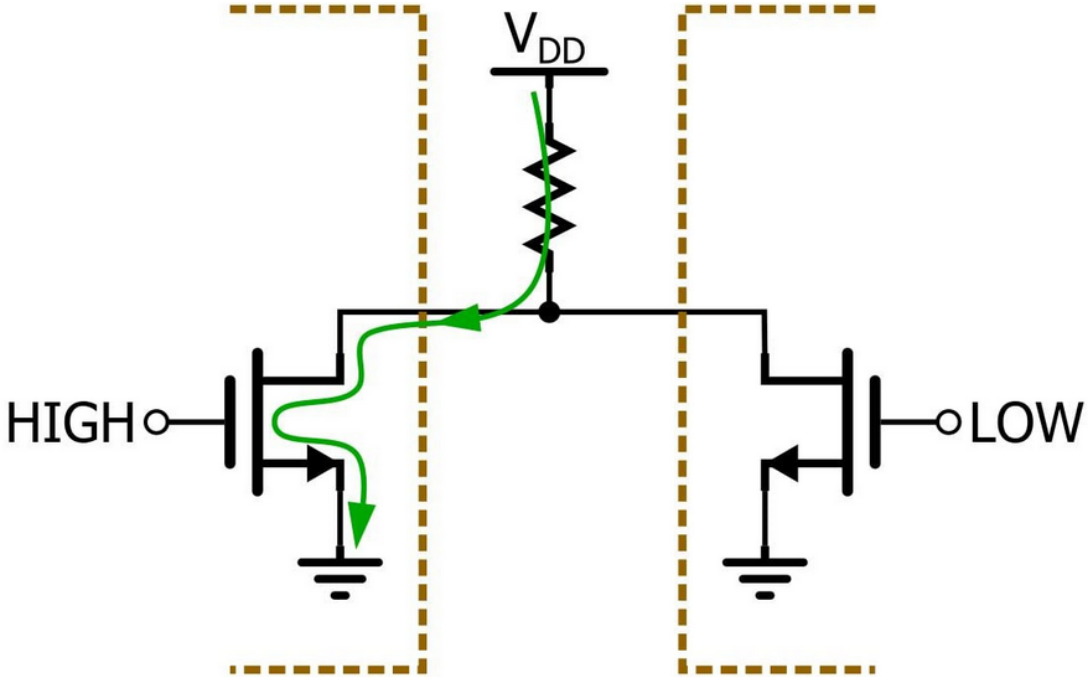
Possible Contention with I^2C

- What if two or more devices, having typical CMOS, inverting output stages, want to send data at the same time?
  - If two devices try to drive the line to the *same* value, either *high* or *low*, then there is no conflict
  - If they try to drive to different values, then we have a *power-ground short circuit!*
    - In the figure below, Q of both gates are connected to the communication (data) line but one is connected to V_{DD} and the other is connected to GND



Possible Contention with I^{2}C – Solution[1]

- Open-drain circuits are used for the I^{2}C bus
  - pMOS transistor replaced by pull-up resistor
- Default state of the communication line is 1
- Any device can only drive output to GND, but not to 1



[1] https://www.allaboutcircuits.com/technical-articles/the-i2c-bus-hardware-implementation-details/

Clock-Stretching

- The open-drain connections used in I2C buses exhibit negative-true wired-OR behavior.
  - An addressed slave device might hold the SCL low as a “not ready” signal
  - Even if the master tries to raise SCL, it will still be low
- Some I2C derivatives limit the amount of time that a slave can hold SCL low

Arbitration

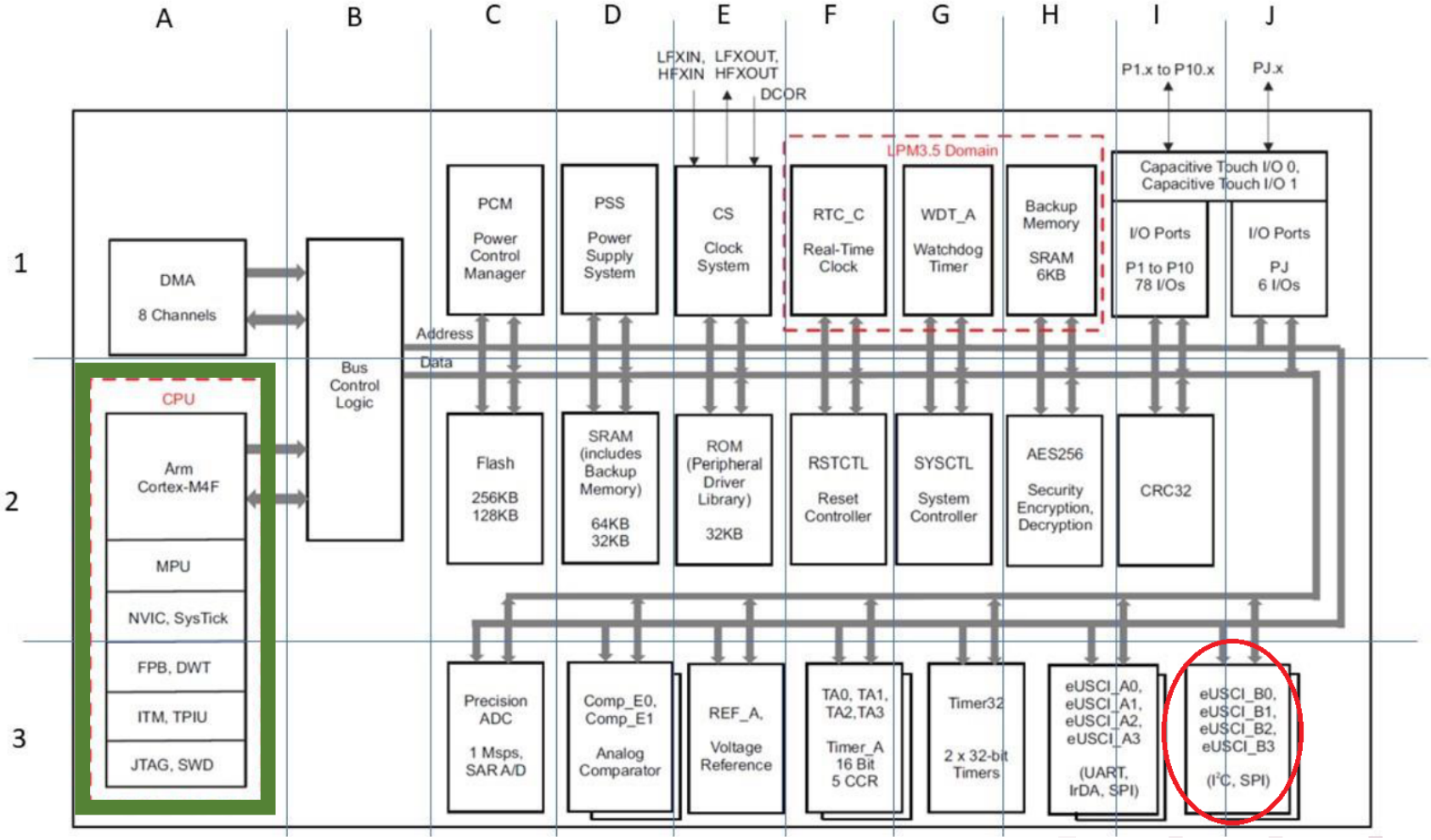
- Sometimes two or more devices will try to transmit on SDA at the same time
- Remember — because of the wired-OR nature of the lines, any device that pulls SDA low will cause SDA to be low
- Arbitration occurs through the process of each device comparing the value on SDA to the value it wants to put on SDA; the "first" device to "notice" a difference "realizes" that another device is transmitting, and so it stops its transmission

Additional Arbitration Discussion

- Protocol
  - Rules by which the two parties agree in order to communicate (start, address, r/w, data, ack, . . ., stop)
- What happens if two masters are trying to address different slaves?
- What happens if two masters are trying to address the same slave?
- What happens if two masters are trying to address the same slave with the same information?

Coming Attractions: Programming I^{2}C on the MSP432

MSP432 Block Diagram



Summary

- I^2C protocol
  - Serial, synchronous, half-duplex communication
  - SCL, SDA, both with open-drain connections
  - Protocol of transmission
  - Arbitration